Revisão: Pilha, Fila e Lista flexíveis em Java

Oct 26, 2026 · @Engenharia Digital

Como usar esta folha

Todos os códigos seguem o estilo das aulas: célula sentinela, `getProximo()` e `setProximo()`, e `equals()` ou `compareTo()` para comparar itens. Eles foram compilados e testados em Java antes de entrar aqui.

Onde fica a sentinela em cada estrutura:

Estrutura	Ponteiros	Sentinela	Vazia quando	Primeiro item real
Pilha	`topo`, `fundo`	no fundo	`fundo == topo`	`topo`
Fila	`frente`, `tras`	na frente	`frente == tras`	`frente.getProximo()`
Lista	`primeiro`, `ultimo`	no início	`primeiro == ultimo`	`primeiro.getProximo()`

Aviso sobre `compareTo`: para usar `compareTo` com um tipo genérico, a classe precisa ser declarada como `Lista<E extends Comparable<E>>`. Com só `Lista<E>`, o Java não compila. Na prova em papel a professora costuma escrever `<E>`; no laboratório, use a forma completa.

Ordem alfabética ou numérica: o mesmo `compareTo` faz as duas coisas. Com `String` ele compara em ordem alfabética e com `Integer` em ordem numérica. `a.compareTo(b)` retorna negativo se `a` vem antes, zero se são iguais e positivo se `a` vem depois.

1. Célula simples e Célula dupla

A célula guarda um item e o endereço da próxima célula. O construtor vazio serve para criar a sentinela.

```
public class Celula<T> {
    private final T item;
    private Celula<T> proximo;

    public Celula() { this(null); }          // usada para a sentinela
    public Celula(T item) {
        this.item = item;
        this.proximo = null;
    }
    public T getItem() { return item; }
    public Celula<T> getProximo() { return proximo; }
    public void setProximo(Celula<T> proximo) { this.proximo = proximo; }
}
```

Na lista duplamente encadeada, a célula também guarda o endereço da célula anterior.

```
public class CelulaDupla<T> {
    private final T item;
    private CelulaDupla<T> anterior;
    private CelulaDupla<T> proximo;

    public CelulaDupla() { this(null); }
    public CelulaDupla(T item) {
        this.item = item;
        this.anterior = null;
        this.proximo = null;
    }
    public T getItem() { return item; }
    public CelulaDupla<T> getAnterior() { return anterior; }
    public void setAnterior(CelulaDupla<T> anterior) { this.anterior =
anterior; }
    public CelulaDupla<T> getProximo() { return proximo; }
    public void setProximo(CelulaDupla<T> proximo) { this.proximo = proximo;
}
}
```

Como `item` é `final`, não dá para trocar o item de uma célula. Para reorganizar a estrutura, você troca as ligações (`setProximo`), não os itens.

2. Pilha flexível

A sentinela fica no fundo e cada célula aponta para a que está embaixo dela. Inserção e remoção acontecem só no `topo`.

```
topo → [X3] → [X2] → [X1] → [sentinela] → null
                             ↑ fundo
```

Métodos principais

```
public class Pilha<E extends Comparable<E>> {
    private Celula<E> topo;
    private Celula<E> fundo;

    public Pilha() {
        Celula<E> sentinela = new Celula<>();
        fundo = topo = sentinela;
    }

    public boolean vazia() {
        return fundo == topo;
    }

    public void empilhar(E item) {
        Celula<E> nova = new Celula<>(item);
        nova.setProximo(topo);   // 1º: nova aponta para o topo atual
        topo = nova;             // 2º: nova vira o topo
    }

    public E desempilhar() {
        if (vazia()) throw new IllegalStateException("Pilha vazia!");
        Celula<E> removida = topo;
        topo = topo.getProximo();
        removida.setProximo(null);
        return removida.getItem();
    }

    public E consultarTopo() {
        if (vazia()) throw new IllegalStateException("Pilha vazia!");
        return topo.getItem();
    }
```

Percorrer sem destruir: imprimir e contar

O `aux` anda do topo até chegar na sentinela (`fundo`). A pilha não muda.

```
    // ordem em que seriam desempilhados (topo -> fundo)
    public void imprimir() {
        Celula<E> aux = topo;
        while (aux != fundo) {
            System.out.print(aux.getItem() + " ");
            aux = aux.getProximo();
        }
        System.out.println();
    }

    public int obterNumeroItens() {
        int cont = 0;
        Celula<E> aux = topo;
        while (aux != fundo) {
            cont++;
            aux = aux.getProximo();
        }
        return cont;
    }
```

Procurar um item (iterativo e recursivo)

```
    public boolean contem(E item) {
        Celula<E> aux = topo;
        while (aux != fundo) {
            if (aux.getItem().equals(item)) return true;
            aux = aux.getProximo();
        }
        return false;
    }

    // versão recursiva + método auxiliar que inicia a recursão
    public boolean contemRecursivo(E item) {
        return contem(item, topo);
    }
    public boolean contem(E item, Celula<E> aux) {
        if (aux == fundo) return false;                 // caso base: acabou
        if (aux.getItem().equals(item)) return true;    // achou
        return contem(item, aux.getProximo());          // procura no resto
    }
```

Imprimir na ordem de inserção (recursivo, exercício 4)

A recursão desce até o fundo primeiro e só imprime na volta. Por isso o primeiro inserido sai primeiro.

```
    public void imprimirOrdemInsercao() {
        imprimirOrdemInsercao(topo);
        System.out.println();
    }
    private void imprimirOrdemInsercao(Celula<E> aux) {
        if (aux != fundo) {
            imprimirOrdemInsercao(aux.getProximo()); // desce primeiro
            System.out.print(aux.getItem() + " ");   // imprime na volta
        }
    }
}
```

Teste: empilhando `maria, ana, joao, carlos, bia`, o `imprimir()` mostra `bia carlos joao ana maria` e o `imprimirOrdemInsercao()` mostra `maria ana joao carlos bia`.

3. Fila flexível

A sentinela fica na frente. Entra por `tras`, sai pela frente, e o primeiro item real é `frente.getProximo()`.

```
frente → [sentinela] → [X1] → [X2] → [X3] → null
                                      ↑ tras
```

Métodos principais

```
public class Fila<E extends Comparable<E>> {
    private Celula<E> frente;
    private Celula<E> tras;

    public Fila() {
        Celula<E> sentinela = new Celula<>();
        frente = tras = sentinela;
    }

    public boolean vazia() {
        return frente == tras;
    }

    public void enfileirar(E item) {
        Celula<E> nova = new Celula<>(item);
        tras.setProximo(nova);
        tras = nova;
    }
```

```
    public E desenfileirar() {
        if (vazia()) throw new IllegalStateException("Fila vazia!");
        Celula<E> primeira = frente.getProximo();
        frente.setProximo(primeira.getProximo());
        if (primeira == tras)      // era o único item
            tras = frente;
        primeira.setProximo(null);
        return primeira.getItem();
    }

    public E consultarPrimeiro() {
        if (vazia()) throw new IllegalStateException("Fila vazia!");
        return frente.getProximo().getItem();
    }
```

Alguns professores escrevem o `desenfileirar` movendo a sentinela (`frente = frente.getProximo()`), e a célula que sai vira a nova sentinela. As duas versões funcionam.

Códigos do quadro: inserirNoInicio e removerFim

```
    public void inserirNoInicio(E item) {
        Celula<E> nova = new Celula<>(item);
        Celula<E> aux = frente.getProximo();
        frente.setProximo(nova);
        nova.setProximo(aux);
        if (frente == tras)      // a fila estava vazia
            tras = nova;
    }

    // remove o último usando uma pilha de apoio
    public E removerFim() {
        Pilha<E> pilha = new Pilha<>();
        while (!vazia())
            pilha.empilhar(desenfileirar());
        E temp = pilha.desempilhar();       // o último da fila
        while (!pilha.vazia())
            inserirNoInicio(pilha.desempilhar());
        return temp;
    }
```

Percorrer: imprimir e contar (recursivo, exercício 4)

Na fila o `aux` começa em `frente.getProximo()` e para quando chega em `null`.

```
    public void imprimir() {
        Celula<E> aux = frente.getProximo();
        while (aux != null) {
            System.out.print(aux.getItem() + " ");
            aux = aux.getProximo();
        }
        System.out.println();
    }

    public int obterNumeroItens() {
        return obterNumeroItens(frente.getProximo());
    }
    public int obterNumeroItens(Celula<E> aux) {
        if (aux == null) return 0;
        return 1 + obterNumeroItens(aux.getProximo());
    }
```

Procurar um item (exercícios 5 e 10)

```
    public boolean contem(E item) {
        return contem(item, frente.getProximo());
    }
    public boolean contem(E item, Celula<E> aux) {
        if (aux == null) return false;
        if (aux.getItem().equals(item)) return true;
        return contem(item, aux.getProximo());
    }

    // quantos itens estão à frente do item procurado
    public int quantosAfrente(E item) {
        return quantosAfrente(frente.getProximo(), item);
    }
    public int quantosAfrente(Celula<E> atual, E item) {
        if (atual == null) throw new IllegalArgumentException("Item não está
na fila!");
        if (atual.getItem().equals(item)) return 0;
        return 1 + quantosAfrente(atual.getProximo(), item);
    }
}
```

Teste: na fila `30 10 50 20 40`, `inserirNoInicio(99)` gera `99 30 10 50 20 40`, e `removerFim()` retorna `40`. Depois disso, `quantosAfrente(50)` retorna `3`.

4. Lista simplesmente encadeada

A lista insere e remove em qualquer posição. O truque é sempre parar na célula **anterior** à posição desejada, porque só ela consegue mudar a ligação.

Métodos principais (versão dos slides)

```
public class Lista<E extends Comparable<E>> {
    private Celula<E> primeiro;
    private Celula<E> ultimo;
    private int tamanho;

    public Lista() {
        Celula<E> sentinela = new Celula<>();
        primeiro = ultimo = sentinela;
        tamanho = 0;
    }

    public boolean vazia() {
        return primeiro == ultimo;
    }

    public void inserir(E novo, int posicao) {
        if (posicao < 0 || posicao > tamanho)
            throw new IndexOutOfBoundsException("Posição inválida!");
        Celula<E> anterior = primeiro;
        for (int i = 0; i < posicao; i++)        // para na célula ANTES da
posição
            anterior = anterior.getProximo();
        Celula<E> novaCelula = new Celula<>(novo);
        Celula<E> proximaCelula = anterior.getProximo();
        anterior.setProximo(novaCelula);
        novaCelula.setProximo(proximaCelula);
        if (posicao == tamanho)                  // inseriu no fim
            ultimo = novaCelula;
        tamanho++;
    }

    public void inserirInicio(E novo) { inserir(novo, 0); }
    public void inserirFinal(E novo)  { inserir(novo, tamanho); }

    public E remover(int posicao) {
        if (vazia()) throw new IllegalStateException("Lista vazia!");
        if (posicao < 0 || posicao >= tamanho)
            throw new IndexOutOfBoundsException("Posição inválida!");
        Celula<E> anterior = primeiro;
```

```
        for (int i = 0; i < posicao; i++)
            anterior = anterior.getProximo();
        Celula<E> celulaRemovida = anterior.getProximo();
        Celula<E> proximaCelula = celulaRemovida.getProximo();
        anterior.setProximo(proximaCelula);
        celulaRemovida.setProximo(null);
        if (celulaRemovida == ultimo)            // removeu o último
            ultimo = anterior;
        tamanho--;
        return celulaRemovida.getItem();
    }

    public void imprimir() {
        Celula<E> aux = primeiro.getProximo();
        while (aux != null) {
            System.out.print(aux.getItem() + " ");
            aux = aux.getProximo();
        }
        System.out.println();
    }
```

Repare na diferença entre os limites: no `inserir` a posição pode ser igual a `tamanho` (inserir no fim), mas no `remover` ela precisa ser menor que `tamanho`.

Procurar um item

```
    // retorna a posição do item, ou -1 se não achar
    public int localizar(E item) {
        Celula<E> aux = primeiro.getProximo();
        int pos = 0;
        while (aux != null) {
            if (aux.getItem().equals(item)) return pos;
            aux = aux.getProximo();
            pos++;
        }
        return -1;
    }

    // recursivo (exercício 4)
    public boolean contem(E item) {
        return contem(item, primeiro.getProximo());
    }
    public boolean contem(E item, Celula<E> aux) {
        if (aux == null) return false;
        if (aux.getItem().equals(item)) return true;
        return contem(item, aux.getProximo());
```

```
    }

    // recursivo (exercício 3): quantas vezes o item aparece
    public int contarQtos(E item) {
        return contarQtos(item, primeiro.getProximo());
    }
    public int contarQtos(E item, Celula<E> aux) {
        if (aux == null) return 0;
        int achou = aux.getItem().equals(item) ? 1 : 0;
        return achou + contarQtos(item, aux.getProximo());
    }
```

Remover todas as ocorrências (exercício 11)

Quando remove, o `anterior` **não anda**, porque a próxima célula pode ser outra ocorrência.

```
    public boolean removerTodos(E item) {
        Celula<E> anterior = primeiro;
        if (this.vazia()) throw new IllegalStateException("Lista vazia!");
        boolean removeu = false;
        for ( ; anterior.getProximo() != null; ) {
            Celula<E> atual = anterior.getProximo();
            if (atual.getItem().equals(item)) {
                anterior.setProximo(atual.getProximo()); // pula a célula
                atual.setProximo(null);
                if (atual == ultimo) ultimo = anterior;
                tamanho--;
                removeu = true;                          // NÃO avança
anterior
            } else {
                anterior = anterior.getProximo();
            }
        }
        return removeu;
    }
```

Teste: na lista `d b a b c b`, `localizar("c")` retorna `4`, `contarQtos("b")` retorna `3`, e `removerTodos("b")` deixa `d a c`.

5. Lista duplamente encadeada

Cada célula aponta para a próxima **e** para a anterior. A vantagem é remover o último sem percorrer a lista (O(1)); a desvantagem é o espaço extra e ter que atualizar 4 ligações em vez de 2.

```
primeiro → [sentinela] ⇄ [X1] ⇄ [X2] ⇄ [X3] → null
                                          ↑ ultimo
```

```
public class ListaDupla<E> {
    private CelulaDupla<E> primeiro;
    private CelulaDupla<E> ultimo;
    private int tamanho;

    public ListaDupla() {
        CelulaDupla<E> sentinela = new CelulaDupla<>();
        primeiro = ultimo = sentinela;
        tamanho = 0;
    }

    public boolean vazia() {
        return primeiro == ultimo;
    }

    public void inserirFinal(E item) {
        CelulaDupla<E> nova = new CelulaDupla<>(item);
        nova.setAnterior(ultimo);
        ultimo.setProximo(nova);
        ultimo = nova;
        tamanho++;
    }

    // O(1): não precisa percorrer, graças ao "anterior"
    public E removerFinal() {
        if (vazia()) throw new IllegalStateException("Lista vazia!");
        CelulaDupla<E> removida = ultimo;
        ultimo = ultimo.getAnterior();
        ultimo.setProximo(null);
        removida.setAnterior(null);
        tamanho--;
        return removida.getItem();
    }

    public void inserir(E item, int posicao) {
        if (posicao < 0 || posicao > tamanho)
            throw new IndexOutOfBoundsException("Posição inválida!");
```

```
        CelulaDupla<E> anterior = primeiro;
        for (int i = 0; i < posicao; i++)
            anterior = anterior.getProximo();
        CelulaDupla<E> nova = new CelulaDupla<>(item);
        CelulaDupla<E> proxima = anterior.getProximo();
        nova.setAnterior(anterior);       // 4 ligações
        nova.setProximo(proxima);
        anterior.setProximo(nova);
        if (proxima != null)
            proxima.setAnterior(nova);
        else
            ultimo = nova;                // inseriu no fim
        tamanho++;
    }

    public E remover(int posicao) {
        if (vazia()) throw new IllegalStateException("Lista vazia!");
        if (posicao < 0 || posicao >= tamanho)
            throw new IndexOutOfBoundsException("Posição inválida!");
        CelulaDupla<E> removida = primeiro.getProximo();
        for (int i = 0; i < posicao; i++)
            removida = removida.getProximo();
        CelulaDupla<E> anterior = removida.getAnterior();
        CelulaDupla<E> proxima = removida.getProximo();
        anterior.setProximo(proxima);
        if (proxima != null)
            proxima.setAnterior(anterior);
        else
            ultimo = anterior;            // removeu o último
        removida.setProximo(null);
        removida.setAnterior(null);
        tamanho--;
        return removida.getItem();
    }

    // de trás para frente, usando "anterior"
    public void imprimirInverso() {
        CelulaDupla<E> aux = ultimo;
        while (aux != primeiro) {
            System.out.print(aux.getItem() + " ");
            aux = aux.getAnterior();
        }
        System.out.println();
    }

    public boolean contem(E item) {
```

```
        CelulaDupla<E> aux = primeiro.getProximo();
        while (aux != null) {
            if (aux.getItem().equals(item)) return true;
            aux = aux.getProximo();
        }
        return false;
    }
}
```

Na lista dupla, o `remover` pode ir direto até a célula que sai, porque ela mesma sabe quem é a anterior. Na lista simples, isso não é possível.

6. Reorganizar: ordenar, inverter e intercalar

Há dois jeitos de reorganizar. **Religando células** (dentro da classe, mexendo em `setProximo`) ou **usando só os métodos públicos** (`empilhar`, `desenfileirar`...) com uma estrutura de apoio. Quando a questão diz "considere apenas os métodos apresentados", use o segundo jeito.

Todos ordenam do menor para o maior. Para ordem decrescente, troque `> 0` por `< 0` (ou `<= 0` por `>= 0`) na comparação.

Inserir já na posição certa (Lista, exercício 10)

Anda até achar o primeiro item maior que o novo e encaixa antes dele.

```
    public void inserirEmOrdem(E item) {
        Celula<E> anterior = primeiro;
        while (anterior.getProximo() != null
               && anterior.getProximo().getItem().compareTo(item) < 0) {
            anterior = anterior.getProximo();
        }
        Celula<E> nova = new Celula<>(item);
        nova.setProximo(anterior.getProximo());
        anterior.setProximo(nova);
        if (nova.getProximo() == null) ultimo = nova;
        tamanho++;
    }
```

Ordenar uma Lista ou Fila inteira (religando células)

Solta a corrente toda e reencaixa célula por célula na posição certa. É o Insertion Sort feito com células. Na Fila o código é igual, trocando `primeiro` por `frente` e `ultimo` por `tras`.

```
    public void ordenar() {
        Celula<E> resto = primeiro.getProximo(); // corrente desordenada
        primeiro.setProximo(null);               // lista fica vazia
        ultimo = primeiro;
        while (resto != null) {
            Celula<E> atual = resto;
            resto = resto.getProximo();
            Celula<E> anterior = primeiro;       // acha onde encaixar
            while (anterior.getProximo() != null
                   &&
anterior.getProximo().getItem().compareTo(atual.getItem()) <= 0) {
                anterior = anterior.getProximo();
            }
            atual.setProximo(anterior.getProximo());
            anterior.setProximo(atual);
            if (atual.getProximo() == null) ultimo = atual;
        }
    }
```

Teste com `String`: a lista `d a c z` vira `a c d z`. Com `Integer`, a fila `99 30 10 50 20` vira `10 20 30 50 99`.

Ordenar uma Pilha (só com empilhar e desempilhar)

Usa uma pilha auxiliar que fica sempre ordenada. Antes de colocar um item nela, devolve para a pilha original tudo que for maior que ele.

```
    // resultado: menor item no topo
    public void ordenar() {
        Pilha<E> aux = new Pilha<>();
        while (!this.vazia()) {
            E atual = this.desempilhar();
            while (!aux.vazia() && aux.consultarTopo().compareTo(atual) > 0)
{
                this.empilhar(aux.desempilhar());
            }
            aux.empilhar(atual);
        }
        while (!aux.vazia()) {          // aux tem o maior no topo;
            this.empilhar(aux.desempilhar()); // voltando, inverte
        }
    }
```

Teste: a pilha com topo `bia carlos joao ana maria` fica com topo `ana bia carlos joao maria`.

Inverter

Pilha e Fila invertem passando por uma estrutura do outro tipo, porque uma é LIFO e a outra FIFO.

```
// na classe Fila
public void inverter() {
    Pilha<E> pilha = new Pilha<>();
    while (!vazia()) pilha.empilhar(desenfileirar());
    while (!pilha.vazia()) enfileirar(pilha.desempilhar());
}

// na classe Pilha
public void inverter() {
    Fila<E> fila = new Fila<>();
    while (!vazia()) fila.enfileirar(desempilhar());
    while (!fila.vazia()) empilhar(fila.desenfileirar());
}
```

Na Lista (exercício 7), dá para inverter só virando as setas, sem estrutura de apoio:

```
public void inverter() {
    Celula<E> anterior = null;
    Celula<E> atual = primeiro.getProximo();
    ultimo = (atual == null) ? primeiro : atual;  // o 1º vira o último
    while (atual != null) {
        Celula<E> proxima = atual.getProximo();   // guarda o resto
        atual.setProximo(anterior);               // vira a seta
        anterior = atual;
        atual = proxima;
    }
    primeiro.setProximo(anterior);
}
```

Intercalar duas filas ordenadas numa Lista (exercício 1)

Compara o primeiro de cada fila e tira o menor. Quando uma acaba, copia o resto da outra. É a mesma ideia da etapa de intercalação do Mergesort.

```
    public Lista(Fila<E> fila1, Fila<E> fila2) {
        this();
        while (!fila1.vazia() && !fila2.vazia()) {
            if
(fila1.consultarPrimeiro().compareTo(fila2.consultarPrimeiro()) <= 0)
                inserirFinal(fila1.desenfileirar());
            else
                inserirFinal(fila2.desenfileirar());
        }
        while (!fila1.vazia()) inserirFinal(fila1.desenfileirar());
        while (!fila2.vazia()) inserirFinal(fila2.desenfileirar());
    }
```

Teste: as filas `1 4 9` e `2 3 10 11` geram a lista `1 2 3 4 9 10 11`.

7. Erros mais comuns em prova

1. **Esquecer de atualizar `ultimo` ou `tras`.** Sempre que inserir no fim, remover o último ou inserir numa estrutura vazia, pergunte: "o ponteiro do fim continua certo?"
2. **Inverter a ordem das ligações.** No `empilhar`, fazer `topo = nova` antes de `nova.setProximo(topo)` perde a pilha inteira. Primeiro liga a nova, depois move o ponteiro.
3. **Começar a percorrer na sentinela.** Na Fila e na Lista, o primeiro item real é `getProximo()` da sentinela. Na Pilha, a sentinela é o ponto de parada (`aux != fundo`).
4. **Usar `==` para comparar itens.** `==` compara endereços. Use `equals()` para igualdade e `compareTo()` para ordem.
5. **Destruir a estrutura quando a questão pede para não destruir.** Para percorrer, use `aux`. Se precisar desempilhar ou desenfileirar, reconstrua a estrutura no final.
6. **Esquecer o caso base na recursão.** Todo método recursivo em células começa com `if (aux == null)` (ou `aux == fundo` na pilha).
7. **Esquecer a exceção de estrutura vazia** em `desempilhar`, `desenfileirar`, `remover` e `consultar`.